

Le bouton de paiement ne se débloque jamais

Une option de compaction du formulaire de paiement, ajoutée le 19 août, viole une règle d'intégration Stripe et provoque une exception non interceptée. Le bouton « Traitement en cours... » reste bloqué indéfiniment sur les cinq flux de paiement.

● CORRIGÉ · EN PRODUCTION DEPUIS LE 20/08

GRAVITÉ Critique — aucun paiement possible pendant l'incident	DÉBUT 19/08/2026 10:16	CAUSE IDENTIFIÉE 20/08/2026 ~00:50
DURÉE ≈ 15 h, résolu côté code	PÉRIMÈTRE 5 flux : boutique, carte, événement, location, reprise de session	COMMIT DÉCLENCHEUR 917b3db

Cause déclenchante

Le commit `917b3db` (19/08, 10:16 — cycle §66, « checkout Stripe compacté ») ajoute quatre options au `PaymentElement` du composant partagé `StripePaymentStep.tsx`. L'une d'elles est la cause directe de l'anomalie :

LA LIGNE À L'ORIGINE DU DÉFAUT

`917b3db` · `StripePaymentStep.tsx`

```
<PaymentElement
  options={{
    layout: 'accordion',
    fields: { billingDetails: { address: 'never' } },
    defaultValues: customerEmail ? { ... } : undefined,
  }}
/>
```

Fixer un champ de `billingDetails` à `'never'` revient à dire à Stripe : « ne demande pas cette donnée à l'utilisateur ». Stripe l'autorise, mais à une condition précise : la donnée omise doit être fournie manuellement à la confirmation, dans `confirmParams.payment_method_data.billing_details`. Cette partie n'a jamais été écrite — `confirmParams` ne contient toujours que `return_url`.

Cette option a été ajoutée pour compacter le formulaire. Le commentaire d'accompagnement affirmait qu'aucun appelant n'attendait d'adresse de facturation : vrai, mais sans rapport. La contrainte ne concerne pas ce qu'attend notre application, mais ce qu'exige Stripe.

Comment une violation de règle devient un blocage permanent

L'erreur d'intégration seule aurait produit un message d'erreur. Ce qui l'a transformée en blocage silencieux et définitif est la conjonction de quatre facteurs, tous dans notre code.

01 Stripe détecte la configuration incohérente

Lors de l'appel à `confirmPayment()`, faute d'`address.country`, Stripe refuse la confirmation.

02 L'erreur est levée, pas retournée

Les `IntegrationError` sont des exceptions. Elles n'arrivent pas dans l'objet `{ error }` vérifié par le code, donc tous les contrôles d'erreur existants sont contournés.

03 Personne n'intercepte l'exception

`handleConfirm` n'avait pas de `try/catch`, et `onClick` ignore la promesse retournée. L'exception sort de la fonction et finit en *unhandled rejection*.

04 L'état du bouton ne revient jamais

Chaque `setIsConfirming(false)` se trouve plus loin dans le flux, désormais inaccessible. Le bouton reste figé sur « Traitement en cours... » jusqu'à ce que l'utilisateur quitte la page — sans aucun message.

PREUVE — CONSOLE DU NAVIGATEUR, /CARD

20/08 · confirmation du diagnostic

```
Uncaught (in promise) IntegrationError: You specified "never" for
fields.billing_details.address when creating the payment Element, but
did not pass confirmParams.payment_method_data.billing_details.address
.country when calling stripe.confirmPayment().
```

Le préfixe `Uncaught (in promise)` confirme à lui seul les points 03 et 04 : l'exception n'a été interceptée par personne.

Chronologie

18/08 · 23:25
b3fc525

Naissance du composant partagé

Les quatre flux de paiement sont unifiés dans `StripePaymentStep.tsx`. Le `PaymentElement` est sans option : fonctionnel. Dès lors, toute modification de ce fichier touche tous les flux à la fois.

19/08 · 10:16
917b3db

Le défaut entre en production

Cycle §66. Quatre options ajoutées en bloc, dont `address: 'never'`. Dès ce moment, la confirmation de paiement lève une exception sur les cinq flux.

19/08 · 18:50
5346f27

Modification sans rapport

Clé Stripe dédiée aux tests e2e. Aucun impact sur le Payment Element — citée seulement pour l'exclure de la bissection.

19/08 · soir

Premier diagnostic : Link soupçonné

La bannière d'inscription Link, apparue au même moment, est prise pour la cause. L'explication est plausible et la désactivation de Link est décidée.

20/08 · 00:39
f953166

Le symptôme est masqué

Cycle §72 : méthodes de paiement restreintes à `card` et timeout de 28 s sur le bouton. Le blocage perpétuel devient une erreur générique après une demi-minute — l'exception sous-jacente reste intacte et invisible.

20/08 · 00:50 env.

Cause réelle identifiée

Le timeout retiré, la bissection des quatre commits du composant isole §66. La console confirme l'`IntegrationError` : Link n'y est pour rien.

20/08 · nuit

Correctif prêt

Cycles §73–§75 terminés et vérifiés par `pnpm typecheck`, livrés sous forme d'archive à la demande explicite de ne pas pousser directement.

20/08 · 01:35–10:35
c076f9c...10bbd7b

Correctif appliqué sur `main`

L'archive livrée est chargée manuellement sur `main`. Vérifié dans cette session par `git fetch` et comparaison directe : les douze fichiers des cycles §73–§75 sont identiques octet pour octet à ceux livrés. **L'incident est clos côté code.**

Impact

- Les cinq points utilisant le composant partagé : checkout boutique, paiement libre /card, réservation d'événement, location de service, reprise d'une session en attente.
- L'utilisateur ne reçoit aucun message : le bouton reste désactivé et la seule issue est de quitter la page.
- Les `PaymentIntent` et `Stripe` sont la trace comptable de l'incident : tentatives interrompues par cette exception, pas des abandons spontanés de clients.
- Entre le 19/08 à 10:16 et le 20/08 à 00:39, le blocage était permanent ; après le timeout de 572, il s'est transformé en erreur générique à 28 secondes, sans paiement abouti pour autant.

Pourquoi cela est passé inaperçu si longtemps

Le symptôme le plus visible n'était pas la cause

La bannière Link est apparue à cause du même commit — le formulaire vidé de ses champs a laissé de la place à l'interface de Link — et a été prise pour l'origine du problème. Deux effets d'une même cause lus comme cause et effet.

Quatre options introduites dans un seul commit

Mise en page, champs, préremplissage et thème ont été ajoutés ensemble. La bissection a isolé le commit, pas la ligne : identifier laquelle des quatre options était responsable a demandé une étape supplémentaire.

La corrélation avec l'email a égaré

Le défaut semblait ne toucher que /card, seul module où l'email est facultative. En réalité le préremplissage de l'email n'y était pour rien : les autres modules présentaient le même problème, simplement pas encore testé.

Le timeout a masqué la preuve

Ajouté de bonne foi comme filet de sécurité, il a remplacé le blocage par un message générique — rendant plus difficile la remontée jusqu'à l'exception. Le retirer a été l'étape qui a permis le diagnostic.

Une analyse menée sur un état de dépôt obsolète

Une phase de l'enquête a conclu que le correctif précédent n'était pas en production, en se fondant sur le dépôt local alors que la branche distante avait été réécrite. Conclusion erronée, corrigée après un `fetch` : l'état déployé doit toujours être vérifié sur le dépôt distant.

Correctif

INTERVENTION	POURQUOI	STATUT
Suppression de <code>address: 'never'</code>	Élimine la cause. Retour au défaut, où Stripe décide lui-même des champs nécessaires et n'exige rien à la confirmation.	FAIT
<code>try/catch</code> dans <code>handleConfirm</code>	Aucune exception, présente ou future, ne peut plus laisser le bouton bloqué sans message.	FAIT
<code>try/catch</code> dans les 5 <code>createIntent</code>	Une coupure réseau pendant l'appel produisait le même blocage, par une autre voie.	FAIT
Suppression du préremplissage email	Passer un email connu déclenche l'authentification Link — origine probable des blocages « trop de tentatives » observés en test.	FAIT
Rétablissement d' <code>automatic_payment_methods</code>	La restriction à <code>card</code> avait été introduite contre Link, qui s'est révélé étranger à l'incident.	FAIT
Légende sur le champ « Pays »	Précise qu'il s'agit du pays de la carte, pas de la position du client.	FAIT
Déploiement en production	Vérifié sur <code>origin/main</code> le 20/08 : les douze fichiers des trois cycles sont présents, identiques octet pour octet à ce qui a été livré.	FAIT
Désactiver Link depuis le Dashboard Stripe	Décision indépendante de l'incident : réduit la friction et le risque de blocage de compte. Non automatisable depuis le code.	À FAIRE

Une action encore ouverte

L'incident est clos côté code : le correctif est sur `main`. Il reste seulement la désactivation manuelle de Link sur le Dashboard Stripe (Settings → Payment Methods → Link) — décision prise pour réduire la friction, indépendante de la cause de cet incident.

Ce qu'il faut changer pour la prochaine fois

– Les options tierces ont des contrats, pas seulement des effets.

Désa 'never'

la

colle

d'un

donn

chez

Strip

créé

l'obl

de

la

four

aille

Avan

de

fixer

une

optio

sur

ou exige en retour

équivalent,

il

faut

chercher

ce

que

cette

valeur

.

- **Un parcours de paiement ne peut pas avoir de branches sans issue.**

Le `try/catch`

désormais

ajouté

n'est

pas

un

correctif

ponctuel

:

c'est

l'exigence

minimale

de

toute

fonction

qui

désactive

un

contrôle

en

attendant

une

promesse.

- **Le composant partagé multiplie le rayon d'action.**

Une `StripePaymentStep.tsx`

ligne

dans

vaut `typecheck`

pour

cing

flux.

Les

mod

de

ce

fichi

méri

une

véri

sur

un

flux

réel,

pas

seul

.

– **La console avant les hypothèses.**

L'erreur
était
visible
et
auto-
explicative
dès
la
première
minute.
Trois
cycles
de
travail
ont
été
consacrés
à
des
hypothèses
qu'une
lecture
de
la
console
aurait
immédiatement
écartées.

– **Un changement par commit, quand il touche une intégration externe.**

Les
quatre
options
de
§66
introduites
séparément
auraient
rendu
la
bissection
immédiate.